연 구 계 획 서

Unreal Engine 5 캐릭터 이동의 클라이언트 자기 예측과 서버 보정 메커니즘 분석

- 리슨 서버 환경에서의 소스 분석과 실측 검증 -

졸업연구및진로 2 | 담당교수 박태순

컴퓨터공학과 | 20011646 | 김민기 | 2026. 10. 15.

1. 연구 배경 및 필요성

멀티플레이 게임에서 네트워크 지연이 생기면 플레이어의 입력이 화면에 늦게 반영되어 반응성이 떨어지고 , 클라이언트가 보는 게임 상태가 서버의 상태와 달라져 일관성도 떨어진다 [1]. 지연은 반응 시간이나 정확도 등의 수행뿐 아니라 조작 용이성, 몰입, 즐거움 같은 플레이 경험에도 나쁜 영향을 준다[2].

이 문제를 줄이는 대표적인 방법이 클라이언트 측 예측이며, 서베이[1]에서는 자기 예측(self-prediction)이라고 부른다. 클라이언트는 서버의 확인을 기다리지 않고 자기 입력의 결과를 먼저 화면에 보여 주고, 나중에 받은 서버의 결과가 이와 다르면 서버가 정한 상태로 돌아간다. 이 방식은 1996 년 QuakeWorld 이후 상용 게임에서 널리 쓰여 왔고[1], 타임워프와 함께 쓰면 왕복 지연이 150ms 까지 늘어나도 수행과 체감 품질에 유의한 영향이 없었다는 결과도 있다[3]. Unreal Engine 에서도 캐릭터 이동 컴포넌트(CharacterMovementComponent, 이하 CMC)에 이 예측과 서버 보정이 기본으로 들어 있다[자료 1].

하지만 서베이[1]에 따르면 자기 예측은 널리 쓰이는 데 비해 평가가 충분히 이루어지지 않았다 . 또 상용 게임에 내장된 보상 기법은 실험 결과를 교란하는 요인으로 여겨져서 [1], 연구자들은 지연을 입력 단계에서 정밀하게 넣거나 [2] 보상 기법을 직접 구현한 실험용 게임에서 효과를 측정해 왔다 [3][4]. 이 때문에 상용 엔진이 예측을 실제로 어떻게 구현했는지, 그 구현이 어떤 조건에서 보정을 일으키는지는 연구되지 않았다. 확인한 범위에서는 Unreal Engine 의 이동 예측과 보정을 분석한 심사 논문을 찾지 못했고, 관련 내용은 공식 문서와 소스 코드에 흩어져 있다[자료 1][자료 2].

2. 연구 목적

이 연구에서는 리슨 서버 환경에서 Unreal Engine 5.5 CMC 의 자기 예측과 서버 보정이 내부에서 어떻게 동작하는지 소스 코드를 따라가며 분석하고, 분석에서 세운 가설을 통제된 측정으로 확인하고자 한다. 연구 질문은 다음과 같다.

Q1. CMC 의 자기 예측에서 클라이언트와 서버의 이동 결과는 어디서 어떻게 어긋나고, 서버는 어떤 조건에서 보정을 시작하는가?

Q2. 보정이 일어나면 클라이언트는 무엇을 되돌리고 다시 실행하며, 보정의 크기와 빈도는 지연, 지터, 패킷 손실, 이동 패턴에 따라 어떻게 달라지는가?

Q3. 리슨 서버의 호스트와 접속 클라이언트는 예측과 보정 경로가 어떻게 다르며, 그 차이는 측정에서 어떻게 나타나는가?

3. 해당 분야의 최근 연구 동향

Liu 등[1]은 심사 논문 85 편을 정리해 지연 보상 기법을 피드백, 예측, 시간 조작, 월드 조정의 4 개 그룹, 11 개 기법으로 분류했고, 자기 예측은 그중 예측 그룹에 속한다. 이 서베이가 소개한 자기 예측 연구 9 편 가운데 4 편은 VR, 터치, 마우스 입력의 예측을, 3 편은 설계 패턴을 다루고 있어 멀티플레이 캐릭터 이동의 예측과 보정을 정량적으로 분석한 연구는 드물다.

Liu 와 Claypool[3]은 Unity 로 만든 자체 FPS 에서 39 명을 대상으로 보상 없음, 자기 예측, 타임워프, 두 기법의 결합이라는 네 조건을 0~150ms 왕복 지연에서 비교했다. 한 기법만으로는 지연의 영향이 다 없어지지 않았지만, 둘을 함께 쓰면 수행과 체감에 대한 영향이 사라졌다. 다만 효과를 수행과 체감으로만 평가했고, 예측이 언제 틀리고 어떻게 바로잡히는지는 다루지 않았다.

Tokey 등[4]은 코너를 사이에 두고 마주치는 교전에서 양쪽의 지연을 따로 조절해, 자신의 지연보다 상대의 지연이 결과를 더 크게 좌우한다는 것을 보였다. 이 실험에서는 클라이언트의 자기 예측만 켜고 서버의 시간 조작은 빼서, 실험 중에 작동하는 보상 기법을 통제했다. Halbhuber 등[2]은 입력 버퍼링으로 지연을 정밀하게 넣은 실험에서 지연이 수행과 대부분의 경험 지표를 떨어뜨린다는 것을 확인했으며, 시점을 1 인칭, 3 인칭, 버드아이 뷰로 바꿔도 지연의 효과는 달라지지 않았다.

최근 연구들은 지연과 보상 기법의 효과를 조건별로 정밀하게 측정하고 있지만, 모두 실험용 게임이나 입력 단계에서 넣은 지연을 사용했다. 상용 엔진의 예측과 보정 구현이 어떤 경로로 오차를 만들고 언제 보정하는지는 아직 분석되지 않았다.

4. 연구 내용 및 방법

4.1 연구 범위

UE 5.5 CMC 에서 플레이어가 직접 조종하는 캐릭터(자율 프록시)의 자기 예측과 서버 보정 과정, 그리고 리슨 서버의 호스트가 접속 클라이언트의 이동을 처리하는 과정을 다룬다. 다른 플레이어 캐릭터의 보간(시뮬레이티드 프록시 스무딩), 게임플레이 어빌리티 시스템의 예측, Network Prediction 및 Mover 플러그인, 물리 복제는 제외하되, 보정 결과의 화면 반영에 스무딩이 관여하면 그 부분은 포함한다.

4.2 연구 절차

(1) 소스 분석 클라이언트가 이동을 저장하고 묶어서 보내는 과정부터 서버가 다시 계산해 오차를 판정하고 보정을 보내는 과정, 클라이언트가 보정을 받아 이동을 다시 실행하는 과정까지 UE 5.5 소스에서 따라가 흐름도로 정리하고, 각 단계를 서베이 [1]의 분류와 대응시킨다. 예를 들어 클라이언트가 보정받은 상태로 돌아가 아직 확인받지 못한 이동을 다시 실행하는 과정 [자료 1]은 상태를 과거로 되돌린 뒤 현재까지 다시 진행한다는 점에서 서베이가 정의한 타임워프 [1]에 대응하지만 수행 주체는 서버가 아닌 클라이언트이다. 공식 문서[자료 1]는 최신 버전 기준이므로 출발점으로만 쓰고, 모든 내용은 5.5 소스에서 다시 확인한다.

(2) 가설 설정 분석한 구조를 바탕으로 측정값으로 맞고 틀림을 판정할 수 있는 가설을 세우고, 측정 전에 확정해 둔다. 지금까지 공식 문서로 확인한 내용에서 세운 가설의 예는 다음과 같다.

- **H1 (Q1)** 서버는 이동마다 다시 계산한 위치와 클라이언트가 보고한 위치의 차이가 허용 기준(MAXPOSITIONERRORSQUARED)을 넘을 때만 보정하며, 보정 사이의 간격은 최소 보정 간격(ClientErrorUpdateRateLimit)보다 짧아지지 않는다.
- **H2 (Q2)** 보정이 일어날 때 다시 실행되는 이동의 수는 왕복 지연 동안 보낸 미확인 이동의 수와 같아 지연에 비례하고, 다시 실행한 직후 남는 오차는 0 에 가깝다.
- **H3 (Q2)** 패킷 손실로 빠진 이동 구간은 다음 이동의 입력으로 대신 계산될 것이므로, 일정한 속도로 직진할 때는 손실이 있어도 보정이 거의 없고 회전이나 정지처럼 입력이 바뀌는 구간에 보정이 몰린다.
- **H4 (Q3)** 서버는 클라이언트의 타임스탬프로 이동 시간을 계산하고 보정은 서버 틱이 끝날 때 보내므로[자료 1], 호스트의 프레임률을 낮추면 접속 클라이언트가 받는 보정의 빈도는 그대로이고 도착 지연만 늘어난다.

(3) 검증 4.3 의 실험으로 가설마다 맞는지 판정하고, 결과가 예상과 다르면 소스로 돌아가 원인을 찾는다.

4.3 실험 환경과 설계

실험은 이 연구를 위해 따로 만든 UE 5.5 프로젝트에서 진행한다. 평지와 기본 3 인칭 캐릭터만 두고 리슨 서버로 접속하며, 프레임률을 고정하고 부하가 큰 렌더링 기능은 끈 채 호스트와 클라이언트를 별도 프로세스로 실행한다. 그래픽보다 프레임률과 낮은 지연을 우선한 전용 실험 환경은 선행 연구[4]에서도 쓰인 방식이다. 네트워크 조건은 엔진에 내장된 패킷 에뮬레이션으로 만들고, 지연 설정값이 편도 기준인지 왕복 기준인지는 예비 실험에서 먼저 확인한다.

비교 기준으로는 CMC 를 쓰지 않고 입력만 서버로 보내 서버에서 움직인 위치를 복제받기만 하는 기준 폰을 만들어, 예측으로 얻는 반응성과 그 대가인 보정을 같은 조건에서 비교한다. 평지에서 정해진 입력만 재생하면 예측이 거의 틀리지 않을 수 있으므로, 서버에서만 이동 속도를 바꾸거나 외력을 가하는 상태 변화를 미리 정한 시점에 넣어 크기와 시점을 알고 있는 오차를 만든다. 입력은 녹화한 시퀀스를 자동 재생하며, H3 확인을 위해 정속 직진, 회전, 불규칙 이동의 세 가지를 쓴다.

실험 변수 9 개의 수준을 모두 조합하면 14,580 개 조건이 되어 측정도 해석도 어렵다. 그렇다고 한 번에 한 변수씩만 바꾸면 H1, H3 처럼 두 변수를 함께 봐야 하는 가설을 확인할 수 없다. 그래서 가설마다 관련 변수만 골라 모든 조합을 측정하도록 실험 블록을 나누었고, 표에 없는 변수는 지터와 패킷 손실은 0, 엔진 설정은 기본값, 프레임률은 기준값으로 고정한다.

블록	대상	바꾸는 변수 (수준)	고정 조건	조건 수
A	H1	오차 허용 기준 3 × 최소 보정 간격 3 (각각 기본값/축소/확대)	지연 100ms, 속도 변경 주입, 직진	9
B	H2	지연 5 (0/50/100/200/400ms)	속도 변경 주입, 직진	5
C	H3	패킷 손실 3 (0/2/5%) × 이동 패턴 3 (직진/회전/불규칙)	지연 100ms, 주입 없음	9
D	H4	호스트 프레임률 2 (기준/절반) × 지연 2 (100/200ms)	속도 변경 주입, 직진	4
E	반응성 비교	예측 방식 2 (기준 폰/CMC) × 지연 5	주입 없음, 직진	10
F	탐색	지터 3 (0/20/50ms) × 주입 종류 2 (속도 변경/외력)	지연 100ms, 직진	6

각 조건은 60 초씩 5 회 반복하고, 블록끼리 겹치는 조건은 한 번만 측정한다. 호스트와 접속 클라이언트의 기록은 매번 함께 남기므로 따로 조건을 나누지 않는다. 지터와 외력은 관련 가설이 없어 탐색 블록(F)에서 관찰 결과만 보고한다.

4.4 측정 지표와 내부 계측

화면에 보이는 위치만으로는 오차가 왜 생겼는지 알 수 없으므로, CMC 를 상속한 계측용 컴포넌트를 만들어 엔진 내부 상태도 함께 기록한다. 기록은 서버 시각을 기준으로 맞추고, 기록 작업이 타이밍에 영향을 주지 않도록 메모리에 모아 두었다가 측정이 끝난 뒤 저장한다. 위치 오차는 캐릭터 캡슐 반지름에 대한 비율로도 함께 적는다.

- 검증 오차와 보정 여부(Q1): 서버가 계산한 위치와 클라이언트가 보고한 위치의 차이, 보정 여부와 시각
- 누적 오차와 잔여 오차(Q2): 서버 확인 전까지 쌓이는 오차와 이동 재실행 직후 남는 오차
- 보정 특성(Q1, Q2): 보정 빈도, 화면상 보정 거리, 재실행된 이동 수, 상태 변화부터 보정까지의 시간
- 반응 지연(Q2): 입력부터 화면에서 캐릭터가 움직이기 시작하기까지의 시간을 기준 폰과 비교

4.5 분석 방법

조건별 평균과 표준편차, 반복 측정 간 분산을 정리하고, 가설마다 소스 분석으로 예상한 값과 측정값을 비교해 맞는지 판정한다. 판정 기준은 결과를 보기 전에 정해 두고, 예상과 다른 결과가 나오면 소스에서 원인을 찾아간 과정까지 기록한다.

5. 예상되는 연구 결과

먼저 CMC 의 예측과 보정 과정을 이동 저장부터 재실행까지 정리한 흐름도와, 각 단계를 학계의 기법 분류에 대응시킨 표를 얻는다. 가설별 검증 결과는 소스 분석으로 세운 모델이 실제 동작을 맞게 예측하는지, 그렇지 않다면 모델에서 무엇이 빠졌는지를 보여 줄 것이다.

또한 지연, 패킷 손실, 이동 패턴, 상태 변화에 따라 보정의 빈도와 크기, 재실행되는 이동 수, 보정 지연이 어떻게 달라지는지와 기준 폰 대비 반응성 이득을 수치로 제시한다. 리슨 서버에서는 호스트와 접속 클라이언트의 처리 차이가 측정에 어떻게 드러나는지 정리한다.

이 연구는 학계가 효과를 측정해 온 자기 예측을 상용 엔진의 구현 수준에서 원인과 조건으로 설명해 본다는 데 의의가 있다.

6. 추진 일정

기간	수행 내용
1~4 주	(완료) 연구 주제 선정, 지연 보상 서베이 및 관련 논문 조사, 연구 방향 구체화
5~6 주	자기 예측 관련 문헌 정리, CMC 예측 및 보정 과정 소스 분석(흐름도, 대응표 작성), 실험용 프로젝트 구축 및 예비 실험
7 주	기준 폰, 계측 컴포넌트, 입력 재생 도구 완성, 가설과 판정 기준 확정
8 주	연구보고서 초안 제출 (배경, 연구 동향, 소스 분석 결과, 가설, 실험 설계, 예비 측정)
9~10 주	조건별 측정 및 가설 검증, 수정본 1, 2 차 반영
11 주	예상과 다른 결과의 원인 추적, 리슨 서버 비교 분석, 수정본 3 차 반영
12 주	가설별 판정과 소스 분석 결과 종합, 최종 수정본 제출
13~15 주	발표자료 작성 및 발표, 연구 보고서 최종 제출

7. 연구 수행상의 고려사항

사람이 직접 조작할 때 생기는 손놀림 차이와 실행 중 부하 변화가 결과에 섞이지 않도록, 입력은 자동으로 재생하고 프레임률을 고정하며 호스트와 클라이언트를 별도 프로세스로 실행한다. 기준 폰과 CMC 는 이동 규칙 자체가 달라서 둘의 비교는 반응 지연과 오차가 생기는 구조로 한정한다.

이 연구의 결과는 UE 5.5 의 CMC 구현에 대한 것이므로 다른 엔진이나 버전에 그대로 적용된다고 보기 어렵다. 또 지표는 서베이[1]가 구분한 반응성과 일관성을 계측값으로 잰 것이며, 플레이어가 느끼는 체감은 측정하지 않는다.

언리얼 구현을 다룬 심사 논문을 찾지 못했기 때문에, 서베이[1]가 개발자 자료를 심사 논문과 따로 다룬 방식을 따라 공식 문서와 소스 코드는 1 차 자료로 구분해 인용한다.

참고문헌

연구 논문

[1] S. Liu, X. Xu, and M. Claypool, "A Survey and Taxonomy of Latency Compensation Techniques for Network Computer Games," ACM Computing Surveys, vol. 54, no. 11s, Article 243, 2022. doi: 10.1145/3519023

[2] D. Halbhuber, P. Schauhuber, V. Schwind, and N. Henze, "The Effects of Latency and In-Game Perspective on Player Performance and Game Experience," Proceedings of the ACM on Human-Computer Interaction, vol. 7, no. CHI PLAY, Article 424, 2023. doi: 10.1145/3611070

[3] S. Liu and M. Claypool, "The Impact of Latency on Target Selection in First-Person Shooter Games," in Proc. 14th ACM Multimedia Systems Conference (MMSys), 2023, pp. 51-61. doi: 10.1145/3587819.3590977

[4] S. S. Tokey, Z. Chen, C. Mettler, D. Tang, B. Boudaoud, J. Kim, J. Spjut, and M. Claypool, "The Effects of Network Latency on the Peeker's Advantage in First-person Shooter Games," in Proc. ACM International Conference on the Foundations of Digital Games (FDG), 2024. doi: 10.1145/3649921.3650002

1 차 자료

[자료 1] Epic Games, "Understanding Networked Movement in the Character Movement Component for Unreal Engine," Unreal Engine Documentation (UE 5.8 문서), https://dev.epicgames.com/documentation/en-us/unreal-engine/understanding-networked-movement-in-the-character-movement-component-for-unreal-engine

[자료 2] Epic Games, Unreal Engine 5.5 소스 코드 (UCharacterMovementComponent, AGameNetworkManager)